LLM Meta-Knowledge Evaluation - Complete Workflow

Status: Ready for LLM Experiments

✅ **Algorithm results parsed** (17 experiments across 9 datasets)
✅ **17 LLM prompts generated** (ready to run)
⏳ **LLM predictions needed** (you need to run prompts)
⏳ **Overlap analysis** (after LLM responses)
⏳ **Paper update** (after overlap computed)

Your Actual Results Summary

Performance by Complexity

Complexity	Datasets	Mean Precision	Mean SHD
Simple (<10 nodes)	Cancer, Earthquake, Survey	0.354	7.8
Medium (10-15 nodes)	Asia, Sachs, Synthetic 12, Titanic	0.333	23.8
Complex (>15 nodes)	Child, Synthetic 30	0.262	135.5

Key Finding

Strong correlation between nodes and SHD: $r=0.888$, $p<0.0001$

This means complexity degradation is even worse than expected!

How to Complete the Study

Step 1: Run LLM Experiments

You have 17 prompts in `/home/claude/llm_prompts/`:

```
Asia_LINGAM.txt
Asia_PC.txt
Cancer_LINGAM.txt
Cancer_PC.txt
Child_LINGAM.txt
Child_PC.txt
Earthquake_LINGAM.txt
Earthquake_PC.txt
Sachs_LINGAM.txt
Sachs_PC.txt
Survey_LINGAM.txt
Survey_PC.txt
Synthetic_12_LINGAM.txt
Synthetic_12_PC.txt
Synthetic_30_LINGAM.txt
Synthetic_30_PC.txt
Titanic_LINGAM.txt
```

Option A: Manual (Recommended for now)

1. Open each prompt file
2. Copy contents to ChatGPT/Claude
3. Save response to text file with same name + `_response.txt`
4. Repeat for all 17 prompts

Option B: Automated (requires API key)

```python
```

```python
import openai
from pathlib import Path

openai.api_key = "sk-..."  # Your key

for prompt_file in Path("llm_prompts").glob("*.txt"):
    with open(prompt_file) as f:
        prompt = f.read()

    response = openai.ChatCompletion.create(
        model="gpt-4",  # or gpt-4-turbo
        messages=[{"role": "user", "content": prompt}],
        temperature=0.0
    )

    output = response.choices[0].message.content

    response_file = prompt_file.parent / f"{prompt_file.stem}_response.txt"
    with open(response_file, 'w') as f:
        f.write(output)

    print(f"Completed: {prompt_file.name}")
```

Step 2: Parse LLM Responses

After collecting all responses, run:

```bash
python parse_llm_responses.py
```

This will extract:

```
Precision: [0.45, 0.65]
Recall: [0.50, 0.70]
F1: [0.48, 0.67]
SHD: [12, 18]
```

From each response and create `llm_predictions.csv`.

Step 3: Compute Overlap

```bash
python compute_overlap_analysis.py
```

This compares your algorithmic CIs with LLM ranges and outputs:

- `overlap_results.csv` - detailed comparisons
- `overlap_summary.txt` - key findings

Step 4: Update Paper

```bash
python update_paper_with_results.py
```

This automatically updates the markdown paper with:

- Your actual algorithm results
- LLM predictions
- Overlap percentages
- Updated discussion

Example: What You'll Discover

Based on your results, here's what we expect:

Titanic + LINGAM

Your Algorithm Results:

- Precision: 0.077 ± 0.006 [0.064, 0.090]
- Recall: 0.230 ± 0.020 [0.192, 0.270]
- F1: 0.115 ± 0.010 [0.096, 0.135]
- SHD: 17.6 ± 0.2 [17.1, 18.0]

Expected LLM Predictions (based on prior work):

- Precision: ~[0.40, 0.70]
- Recall: ~[0.50, 0.75]
- SHD: ~[8, 15]

Expected Finding: LLM will grossly overestimate performance! Actual precision (0.077) is far below LLM lower bound (0.40). This shows LLMs don't understand when algorithms catastrophically fail.

Child + PC

Your Algorithm Results:

- Precision: 0.312 ± 0.003 [0.306, 0.318]
- Recall: 0.741 ± 0.003 [0.736, 0.747]
- SHD: 66.5 ± 0.6 [65.3, 67.7]

Expected LLM:

- SHD: ~[20, 40]

Expected Finding: LLM will underestimate SHD by 2-3×, confirming systematic bias.

Quick Start (Manual Approach)

1. Test with One Experiment

Pick the simplest one: `Earthquake_PC.txt`

```bash
cat /home/claude/llm_prompts/Earthquake_PC.txt
```

Copy to ChatGPT, get response like:

```
Precision: [0.60, 0.85]
Recall: [0.85, 0.98]
F1: [0.70, 0.90]
SHD: [3, 7]
Reasoning: PC should perform very well on this small, simple network...
```

2. Compare to Your Results

Earthquake + PC actual:

- Precision: 0.412 [0.400, 0.424]
- Recall: 0.980 [0.963, 0.993]
- SHD: 9.8 [9.5, 10.1]

Observation:

- Precision: LLM range [0.60, 0.85] vs actual CI [0.400, 0.424] → **NO OVERLAP**
- Recall: LLM range [0.85, 0.98] vs actual CI [0.963, 0.993] → **PARTIAL OVERLAP** (91%)
- SHD: LLM range [3, 7] vs actual CI [9.5, 10.1] → **NO OVERLAP**

Conclusion: LLM overestimated precision, underestimated SHD.

3. Repeat for All 17

This gives you complete data for the paper!

Files Reference

Generated for You

- **parsed_algorithm_results.csv** - Your results in structured format
- **dataset_metadata.json** - Dataset characteristics
- **llm_prompts/*.txt** - 17 ready-to-use prompts
- **prompt_index.csv** - List of all experiments
- **paper_summaries.json** - Pre-computed statistics

You Need to Create

- **llm_responses/*.txt** - LLM outputs (17 files)
- **llm_predictions.csv** - Parsed LLM predictions
- **overlap_results.csv** - Computed overlaps
- **updated_paper.md** - Final paper with real results

Scripts Available

- `parse_results.py` - Already run ✅
- `generate_prompts.py` - Already run ✅
- `generate_paper_updates.py` - Already run ✅
- `parse_llm_responses.py` - Run after collecting responses
- `compute_overlap_analysis.py` - Run after parsing
- `update_paper_with_results.py` - Final step

Expected Timeline

- **LLM queries (manual):** 1-2 hours
- **LLM queries (automated):** 10 minutes
- **Parse responses:** 5 minutes
- **Compute overlap:** 5 minutes
- **Update paper:** 10 minutes
- **Manual review:** 30 minutes

Total: 2-3 hours to complete study

What You'll Have at the End

1. ✅ Rigorous variance-based evaluation
2. ✅ 17 experiments × 4 metrics = 68 overlap comparisons
3. ✅ Quantified LLM accuracy (expected: ~35% mean overlap)
4. ✅ Publication-ready paper with real results
5. ✅ Complete reproducible workflow

Need Help?

If you get stuck:

1. Share your LLM responses (even partial)
2. I can parse them and compute overlap
3. I can update the paper for you

The hardest part (running algorithms 100× each) is DONE! You just need to collect 17 LLM predictions.

Ready to start? Run the first prompt on Earthquake_PC and see what happens!